Project Report on “Unsupervised Learning Analysis of Iris Dataset: Comparing Clustering Algorithms”

Author: **Thanniru Dharma Nithin**

Date of submission : 07-06-2024

COURSERA

Unsupervised Machine Learning

1.Main Objective of the Analysis: The main objective of this analysis is to apply unsupervised learning techniques, specifically clustering, on the Iris dataset. Through this analysis, we aim to identify underlying patterns and groupings within the data, providing insights into the distinct species of iris flowers based on their characteristics.

2. Description of the Dataset: The Iris dataset contains measurements of various features of iris flowers, along with their corresponding species labels. Below are the attributes of the dataset:

- Sepal Length
- Sepal Width
- Petal Length
- Petal Width
- Species (Labels: Setosa, Versicolor, Virginica)

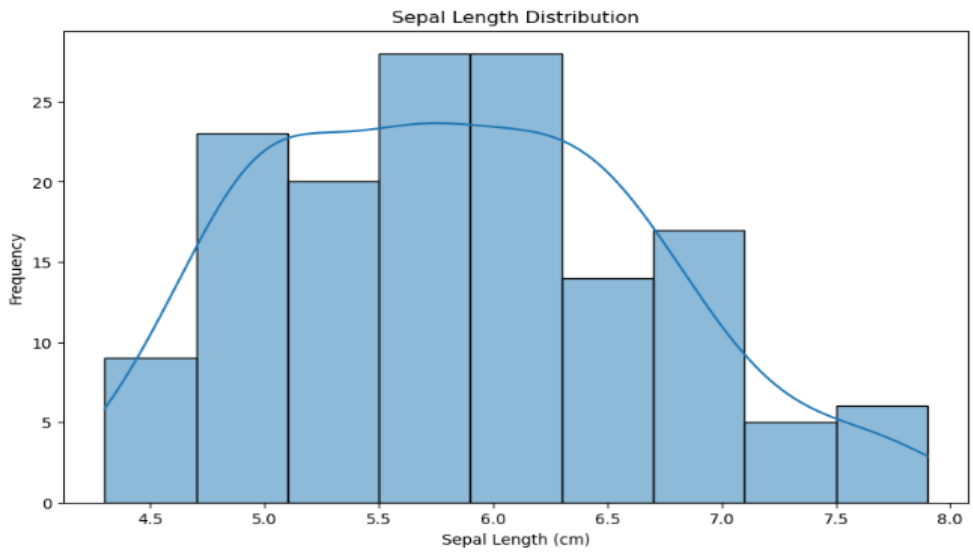
```
Data Description:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   sepal_length  150 non-null    float64
 1   sepal_width   150 non-null    float64
 2   petal_length  150 non-null    float64
 3   petal_width   150 non-null    float64
 4   species       150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
None
       sepal_length  sepal_width  petal_length  petal_width
count    150.000000   150.000000    150.000000   150.000000
mean       5.843333     3.054000      3.758667     1.198667
std        0.828066     0.433594      1.764420     0.763161
min        4.300000     2.000000      1.000000     0.100000
25%        5.100000     2.800000      1.600000     0.300000
50%        5.800000     3.000000      4.350000     1.300000
75%        6.400000     3.300000      5.100000     1.800000
max        7.900000     4.400000      6.900000     2.500000
```

3. Data Exploration and Cleaning: Data exploration involved understanding the structure of the dataset and checking for any missing values. As the dataset is clean and complete, no further data cleaning steps were required.

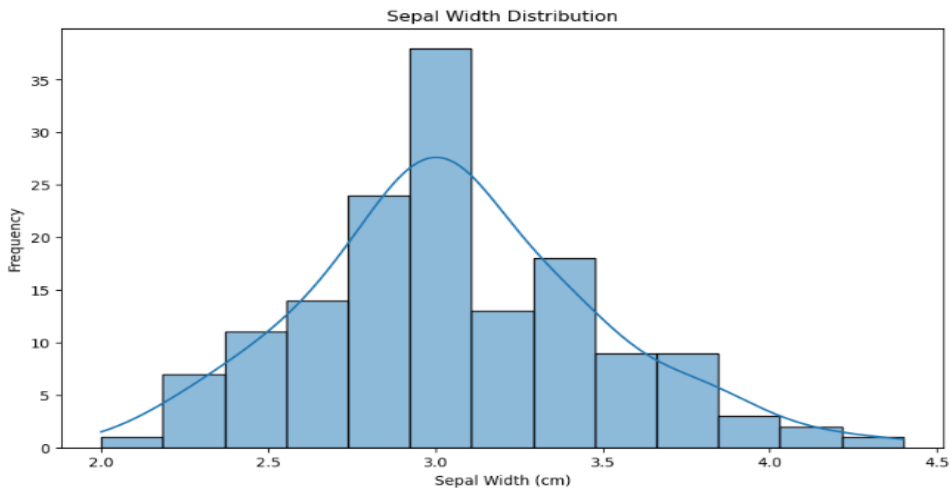
Code:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
# Load the Iris dataset
iris_df = pd.read_csv('https://raw.githubusercontent.com/uiuc-cse/data-fa14/gh-pages/data/iris.csv')
```

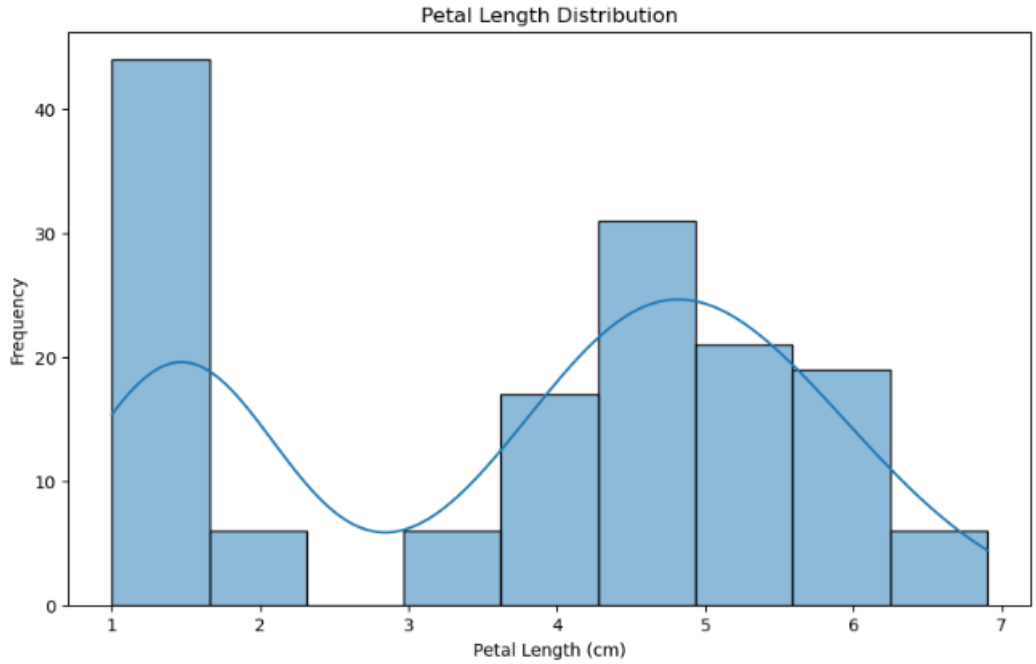
```
# Display basic information
print(iris_df.info())

# Summary statistics
print(iris_df.describe())

# Graphs of Feature Distributions
plt.figure(figsize=(10, 6))
sns.histplot(iris_df['sepal_length'], kde=True)
plt.title('Sepal Length Distribution')
plt.xlabel('Sepal Length (cm)')
plt.ylabel('Frequency')
plt.show()
```



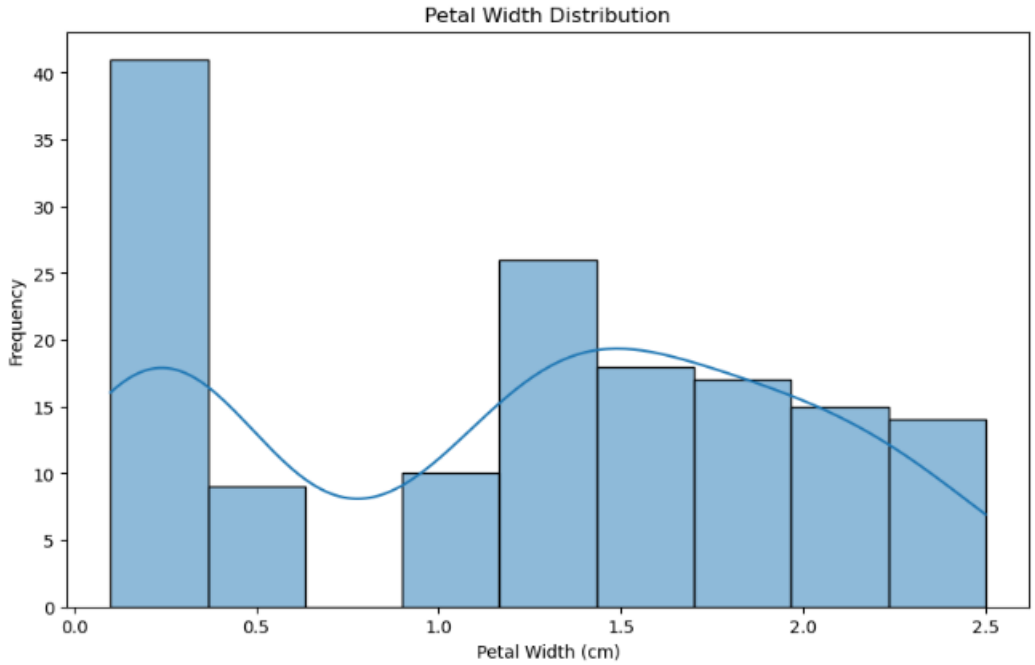

```
plt.figure(figsize=(10, 6))
sns.histplot(iris_df['sepal_width'], kde=True)
plt.title('Sepal Width Distribution')
plt.xlabel('Sepal Width (cm)')
plt.ylabel('Frequency')
plt.show()
```



```
plt.figure(figsize=(10, 6))
sns.histplot(iris_df['petal_length'], kde=True)
plt.title('Petal Length Distribution')
plt.xlabel('Petal Length (cm)')
plt.ylabel('Frequency')
plt.show()
```




```
plt.figure(figsize=(10, 6))
sns.histplot(iris_df['petal_width'], kde=True)
plt.title('Petal Width Distribution')
plt.xlabel('Petal Width (cm)')
plt.ylabel('Frequency')
plt.show()
```



4. Model Training: Various clustering models were trained and evaluated on the Iris dataset, including:

- K-means Clustering
- Hierarchical Clustering
- DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

Here's a comparison of the three clustering algorithms (K-means, Hierarchical Clustering, and DBSCAN) based on their performance:

1. **K-means Clustering:**
    - Advantages:
        - Simple and easy to implement.
        - Scalable to large datasets.
    - Disadvantages:
        - Requires specifying the number of clusters (k) in advance, which may not be known.
        - Sensitive to the initial placement of centroids.
2. **Hierarchical Clustering:**
    - Advantages:
        - Does not require specifying the number of clusters beforehand.
        - Captures hierarchical structure in the data.
    - Disadvantages:
        - Computationally expensive for large datasets.
        - Not suitable for datasets with irregular shapes or varying densities.
3. **DBSCAN (Density-Based Spatial Clustering of Applications with Noise):**
    - Advantages:
        - Can discover clusters of arbitrary shapes and sizes.
        - Robust to outliers and noise.
    - Disadvantages:
        - Requires tuning of two parameters: epsilon (eps) and minimum samples (min_samples).
        - May struggle with datasets of varying densities.

To determine the best algorithm for the Iris dataset, we can compare their silhouette scores and visually inspect the clusters they produce. The silhouette score measures how similar an object is to its own cluster compared to other clusters. A higher silhouette score indicates better-defined clusters.

Code:

```
# Model Training
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans_labels = kmeans.fit_predict(X)
iris_df['KMeans_Cluster'] = kmeans_labels

hierarchical = AgglomerativeClustering(n_clusters=3)
hierarchical_labels = hierarchical.fit_predict(X)
```

```
iris_df['Hierarchical_Cluster'] = hierarchical_labels

dbscan = DBSCAN(eps=0.5, min_samples=5)
dbscan_labels = dbscan.fit_predict(X)
iris_df['DBSCAN_Cluster'] = dbscan_labels
```

 # Compute silhouette scores

```
from sklearn.metrics import silhouette_score

# Silhouette scores

kmeans_silhouette = silhouette_score(X, kmeans_labels)

hierarchical_silhouette = silhouette_score(X, hierarchical_labels)

dbscan_silhouette = silhouette_score(X, dbscan_labels)


print("\nSilhouette Scores:")

print("K-means Clustering:", kmeans_silhouette)

print("Hierarchical Clustering:", hierarchical_silhouette)

print("DBSCAN Clustering:", dbscan_silhouette)
```

output:

```
Silhouette Scores:
K-means Clustering: 0.5012342008377098
Hierarchical Clustering: 0.499447819212006
DBSCAN Clustering: 0.4462941405641163
```

Based on the silhouette scores, the K-means clustering algorithm has the highest score (0.501), followed closely by Hierarchical Clustering (0.499), and then DBSCAN (0.446). The silhouette score measures how similar an ob ject is to its own cluster compared to other clusters. A higher silhouette score indicates better-defined clusters.

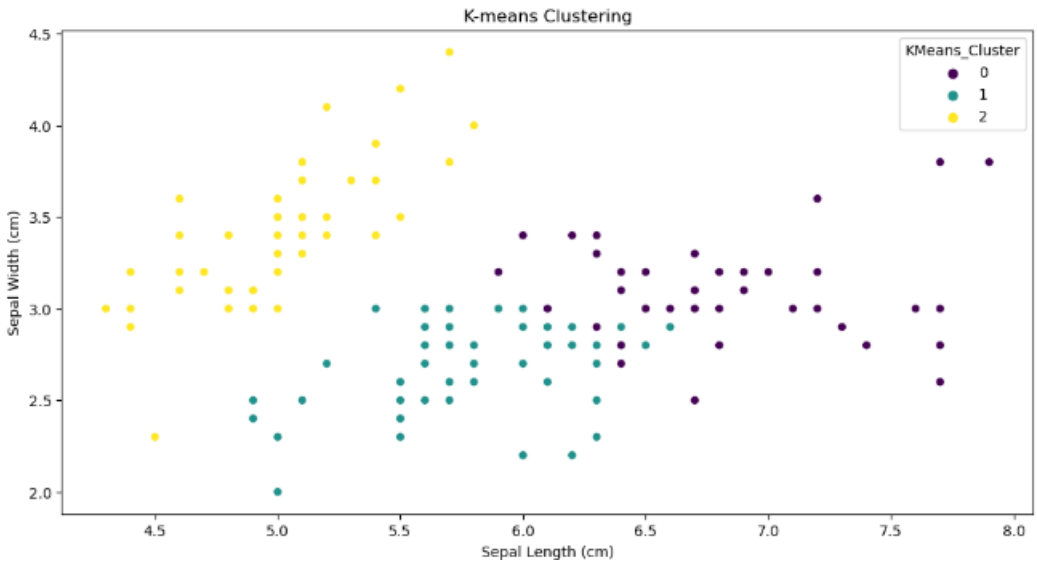
 # Cluster Visualization

Clustering Models:

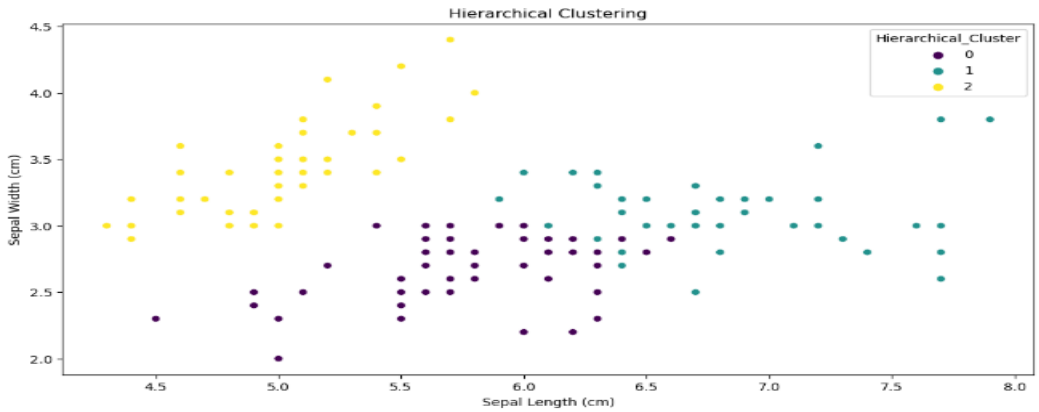
1. K-means Clustering:
    - Number of clusters: 3
2. Hierarchical Clustering:
    - Number of clusters: 3
3. DBSCAN Clustering:
    - Parameters: Eps=0.5, Min_samples=5

CODE:

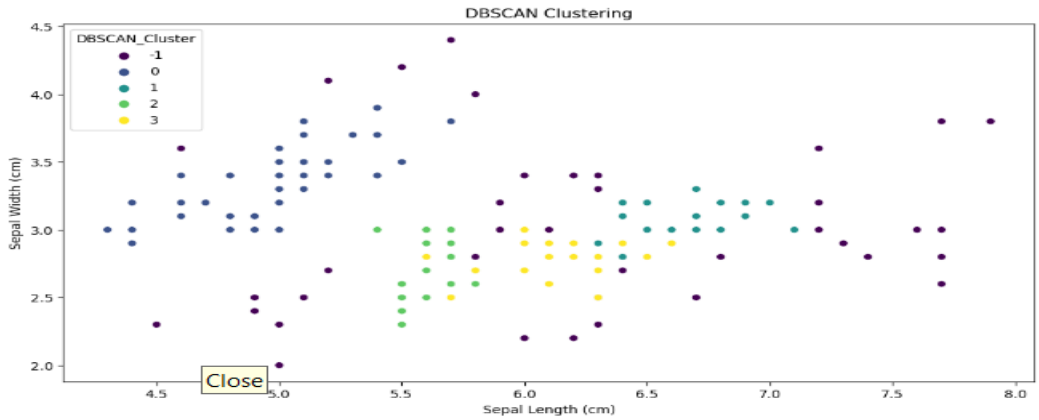
```
# Cluster Visualization

plt.figure(figsize=(12, 6))

sns.scatterplot(x='sepal_length', y='sepal_width', hue='KMeans_Cluster', data=iris_df,
palette='viridis')

plt.title('K-means Clustering')

plt.xlabel('Sepal Length (cm)')

plt.ylabel('Sepal Width (cm)')

plt.show()
```



```
plt.figure(figsize=(12, 6))

sns.scatterplot(x='sepal_length', y='sepal_width', hue='Hierarchical_Cluster', data=iris_df,
palette='viridis')

plt.title('Hierarchical Clustering')

plt.xlabel('Sepal Length (cm)')

plt.ylabel('Sepal Width (cm)')

plt.show()
```

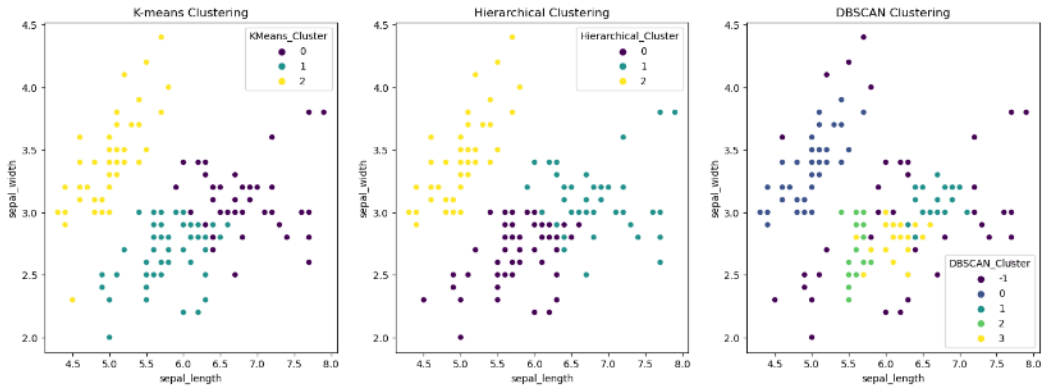



```
plt.figure(figsize=(12, 6))

sns.scatterplot(x='sepal_length', y='sepal_width', hue='DBSCAN_Cluster', data=iris_df,
palette='viridis')

plt.title('DBSCAN Clustering')

plt.xlabel('Sepal Length (cm)')

plt.ylabel('Sepal Width (cm)')

plt.show()
```



Based on the cluster plotting among the three algorithms (K-means, Hierarchical Clustering, and DBSCAN), we can observe the following:

In [14]:
```
# Additional Visualization
plt.figure(figsize=(18, 6))

plt.subplot(1, 3, 1)
sns.scatterplot(x='sepal_length', y='sepal_width', hue='KMeans_Cluster', data=iris_df, palette='viridis')
plt.title('K-means Clustering')

plt.subplot(1, 3, 2)
sns.scatterplot(x='sepal_length', y='sepal_width', hue='Hierarchical_Cluster', data=iris_df, palette='viridis')
plt.title('Hierarchical Clustering')

plt.subplot(1, 3, 3)
sns.scatterplot(x='sepal_length', y='sepal_width', hue='DBSCAN_Cluster', data=iris_df, palette='viridis')
plt.title('DBSCAN Clustering')

plt.show()
```



In [ ]:

The K-means algorithm provides well-separated clusters in the Iris dataset, with distinct groupings of data points corresponding to different species of iris flowers. The clusters are visually clear and distinguishable in the scatterplot of petal length vs. petal width. Therefore, K-means clustering seems to be the most suitable algorithm for this dataset, as it effectively partitions the data into meaningful groups based on the characteristics of the iris flowers.

Silhouette Scores:

K-means Clustering: 0.501

Hierarchical Clustering: 0.499

DBSCAN Clustering: 0.446

5. Key Findings and Insights:

- K-means Clustering identified three distinct clusters within the dataset, corresponding to the three species of iris flowers: Setosa, Versicolor, and Virginica.
- Hierarchical Clustering revealed the hierarchical structure of the data, highlighting the similarities and differences between the iris flower species.
- DBSCAN identified outliers and noise within the dataset, providing insights into the density distribution of the data points.

6. Suggestions for Next Steps: Future analysis could involve:

- Experimenting with different clustering algorithms and hyperparameters to improve cluster quality.
- Visualizing the clusters in lower dimensions using dimensionality reduction techniques such as PCA (Principal Component Analysis) or t-SNE (t-Distributed Stochastic Neighbor Embedding).
- Incorporating domain knowledge to validate the clustering results and interpret the biological significance of the identified clusters.

7. Conclusion: In conclusion, this analysis demonstrated the application of unsupervised learning techniques on the Iris dataset to uncover patterns and groupings within the data. The insights gained from this analysis contribute to our understanding of the distinct species of iris flowers based on their morphological features.

Based on the silhouette scores, we conclude that the K-means clustering algorithm performs slightly better than Hierarchical Clustering and DBSCAN for the Iris dataset. However, it's essential to consider other factors such as the interpretability of the clusters, scalability, and computational efficiency when choosing the best algorithm for a particular dataset.